

1DL301 Group 79 Database Design I - Milestone 1

Group members:

William Min: william.shaye-min.8672@student.uu.se

Upali Bera: upali.bera.4599@student.uu.se

Simen Odden Hansen: simenodden.hansen.6439@student.uu.se

Rishi Maniktala: rishi.maniktala.7931@student.uu.se

Matthias Klaushofer: matthias.klaushofer.0843@student.uu.se

Status:

Milestone 1 - Finished

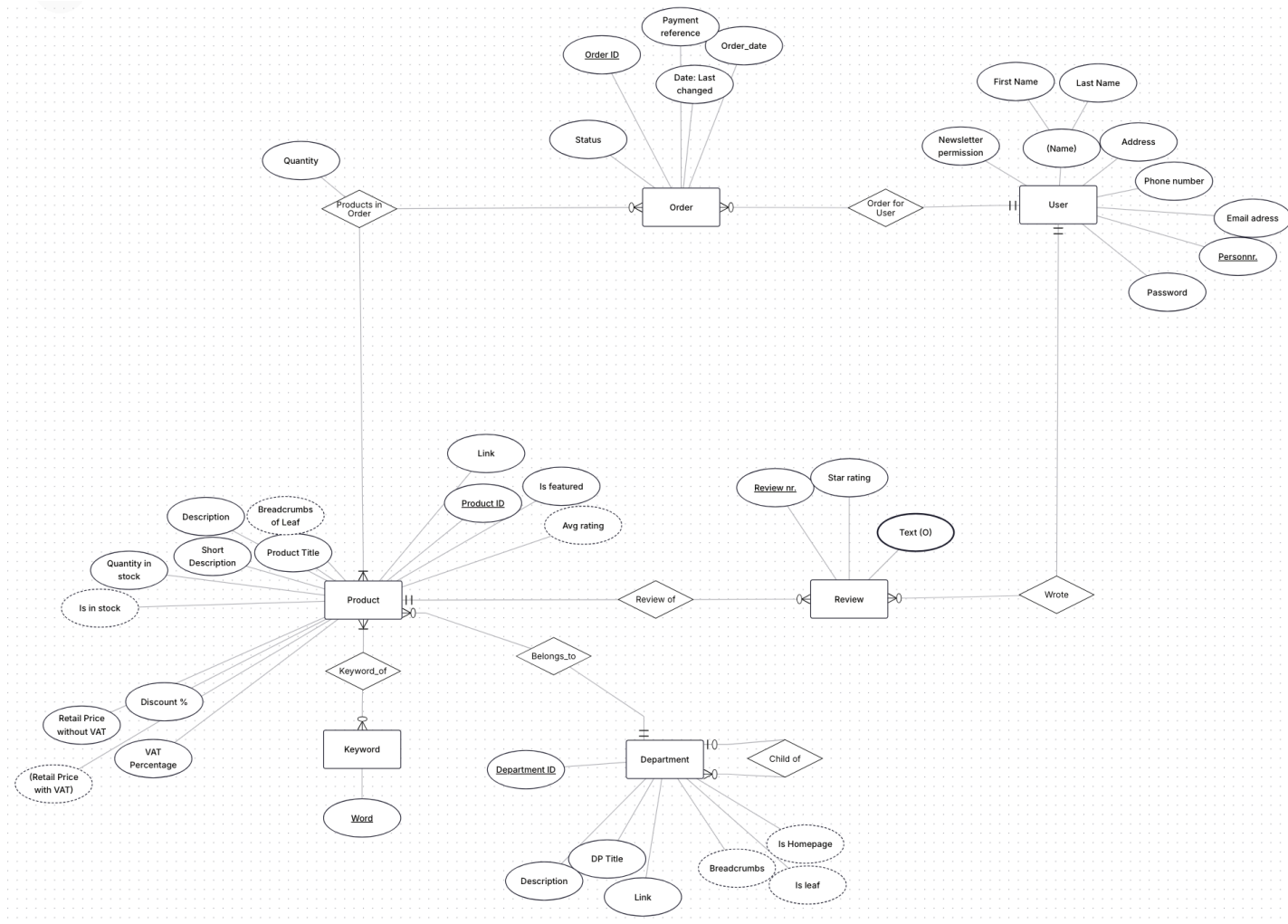
Milestone 2 - Finished

Milestone 3 - Finished

Milestone 4 - Finished

Milestone 1

ER Diagram:



Discussion of ER Diagram:

Departments:

The department schema contains some simple attributes: DP Title, Description, Link, and its primary key, Department ID.

The departments' hierarchical structure is perhaps the most conceptually difficult part to model. To do this, we used a recursive relationship representing the (direct) parent-child relationship for departments. Every department can have 0 or more children, but it must be the child of exactly one department (unless it is the unique root ("home") department). Note that there is a logical constraint which we cannot model in ER that we do not have cyclic relationships where a parent is the child of its child.

From this relation we can derive the breadcrumbs attribute by recursively searching for a department's parent until we reach a department with no parent. We derive "Is Leaf" by checking if a department has no children. These derived attributes could actually be stored in the department table, or they could be computed on the fly. If they are stored, there would need to be logic on when to update them(consider if a parent department name was altered). The ER model is agnostic to these implementation decisions.

We derive "Is Homepage" by checking if a department has no parent. Our root department "Home" will contain data for the homepage. In its "description" we house the homepage's welcome text.

A department can participate in the "Belongs to" relationship with 0 or more products. This represents that those products belong to that department.

Homepage data:

This section does not describe an entity itself, but instead the process to acquire the data necessary to render the homepage, referencing other entities in our database.

As mentioned in departments, the homepage's welcome text will be stored in the description of the "home" department.

We will get the top-level departments as the (direct) children of the home department (the department where "is Homepage" is true). Those departments will contain the relevant attributes for the homepage (title, short description, link)

We will get featured products by observing the `is_featured` attribute of the `products` table. Those products will contain the relevant attributes for the homepage (title, short description, retail price, link)

Products:

A product has the simple attributes: product id(primary ID), product title, description, short description, price without VAT, discount percentage, VAT percentage, quantity in stock, featured status for homepage inclusion and a direct link for navigation. From these attributes, we can also derive the price with VAT and the boolean of whether the item is in stock. We choose not to make the discount percentage optional because an item not on sale can be represented with a discount percentage of zero.

Every product can be related to zero or more reviews with the “review of” relationship, where each review includes a star rating and (optionally) text feedback from a user. A product’s average rating attribute is derived from averaging all the star ratings for reviews related to a given product.

Products must participate in the “Belongs To” relation with a department. Every product must belong to exactly one department. Products may only belong to a department that is a leaf department (one that has no children). This constraint cannot be represented in an ER diagram.

From this relation, we can derive the attribute “Breadcrumbs of leaf,” which represents the breadcrumbs of the (leaf) department that the product belongs to.

Products can also participate in 0 or more instances of the “Products in order” relation, indicating that a particular order contains that product (with a quantity that is an attribute of the relation).

To store data related to keyword recommendations, for each product, we store related keywords in the `Keyword_of` relation. To generate keyword-related products for a given product, we can find all keywords related to a product, then find all products related to that keyword.

Keyword:

A simple entity with a unique primary key representing the keyword. We don’t need an additional primary key id because every instance should represent a distinct word. Every keyword must be a “keyword of” at least one product; otherwise, there would be no reason to store this keyword.

Users:

The user schema has the attributes: *Personnr* (Primary Key), *Name* (consists of First Name and Last Name), *Address*, *Phone Number*, *Email Address* (unique), *Password* and *Newsletter Permission*.

The Swedish Personal Identity Number (Personnr) will serve as the primary key for the user entity. Since this number is a unique and established identifier for every person in Sweden, it is perfectly suitable as a unique identifier for each registered user. The Name attribute consists of first and last names as separate inputs. Address, Phone Number and e-mail address are additional informational attributes, where an e-mail address has to be unique since it will be used to log into a user account. Additionally, we do not want multiple accounts with the same e-mail address. The newsletter permission attribute is a Boolean, and it shows if a user has agreed to receive the store's newsletter.

The user entity is connected to the order entity through the relationship "Order for user." Each user can be related to zero or more orders.

The user entity is connected to the review entity through the relationship "Wrote", which specifies that each user can be related to zero or more reviews.

Review:

The Review entity has three attributes: Review Number (primary key), Text(optional), and Star Rating. The Review Number uniquely identifies each review. Users can assign a Star Rating from 1 to 5, where 1 indicates a poor product and 5 indicates an excellent product, and they can also provide a longer written evaluation in the Text field.

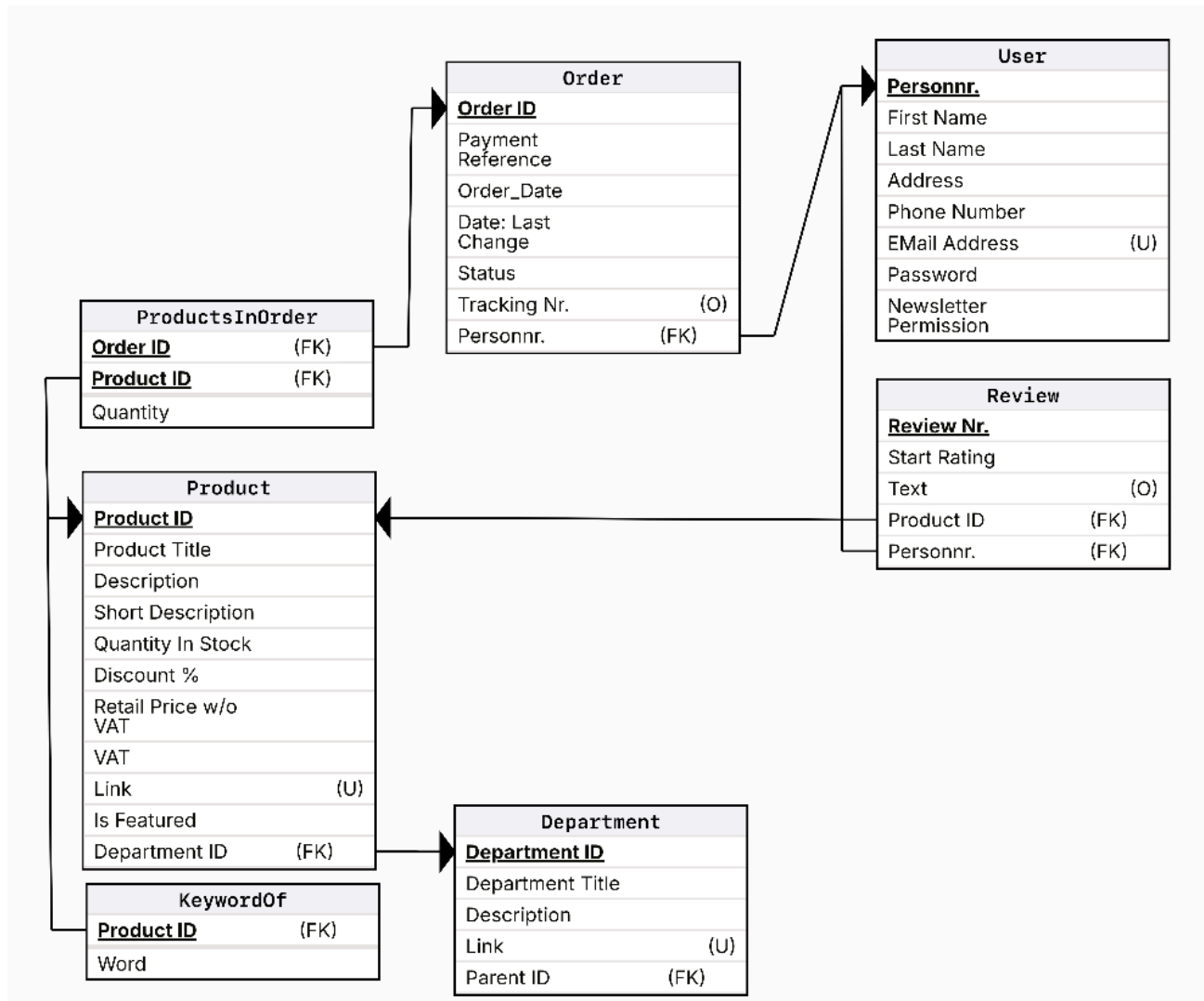
Review has a **one and only one relation** with the user entity, which means each review is associated with exactly one user and another **one and only one relation** with the product entity, which states that each review written by a user can only be related to a single product.

Order:

Our order entity is defined by the primary key *Order ID* and the attributes *Payment reference*, *Status*, *Order date*, *Date last changed*(when the order was last changed), and the optional *Tracking Number*-attribute. We will identify each order using Order_ID. One order has to be connected to one user, and can only be connected to one user at most.

An order is connected to a Product by "products in order", where it must have at least one product, and there can be multiple products in one order. This relation also has a *Quantity*-attribute where the quantities of each product in an order can be specified in case the user wants to order multiple of some products.

Milestone 2



Converting to relational model:

To convert our ER model to a relational model, we describe each entity with a table, with a column for each attribute. We describe 1:N relations by adding a column to the entity on the N side storing the foreign key of the entity on the many side. (if N=1, the choice is arbitrary). For M:N relations, we create a table whose primary key is a member of the cartesian product of the two entities' primary keys.

1st Normal Form

The only nonatomic attribute in our ER model was name, a composite attribute consisting of two others(firstname and lastname). To comply with 1NF, we split that into its components and store each component as its own attribute.

Now, all our attributes are atomic, thus the schema is 1NF.

2nd Normal Form

Here, we just check that every attribute depends upon every part of the primary key. For our tables describing entities, this is very clear because every attribute depends on which instance we are talking about. (an employee's name depends on his/her employee id). Every primary key is a single atomic value so a nonprimal attribute will depend fully on it.

Now, let's check 2NF for our tables describing relations.

For ProductsInOrder, the only non-primal attribute, quantity, is indeed dependent on both parts of the primary key. (The quantity cannot simply be determined by an order id, or a product id, but instead the combination of both).

For KeywordOf and DepartmentRelation, there are no non-primal attributes, so 2NF holds vacuously.

So, no nonprimal attribute depends upon a proper subset of the attributes making up the candidate key, so 2NF is satisfied.

3rd Normal Form

We do not include any of our derived attributes because they involve a transitive dependency (in the form: key -> non key -> derived attribute).

In creating our ER diagram, we took care to mark any attribute involving a transitive dependency (where the intermediate value form was non-key) as derived so that we could exclude it from our relational diagram and avoid violating 3NF.

Later, when using this database, we can construct SQL queries to compute the values of these derived attributes when they are needed.

Datatypes

We also took the effort to declare the datatypes for the attributes.

Columns/Data Types for User:

	Name	Data Type	Data Type Size
	Personnr.	VARCHAR(n)	12
	First Name	VARCHAR(n)	40
	Last Name	VARCHAR(n)	40
	Address	VARCHAR(n)	150
	Phone Number	VARCHAR(n)	15
	EMail Address	VARCHAR(n)	100
	Password	VARCHAR(n)	255
	Newsletter Permissio	Custom	BOOLE...

Columns/Data Types for Review:

	Name	Data Type	Data Type Size
	Review Nr.	VARCHAR(n)	12
	Start Rating	INT	
	Text	VARCHAR(n)	500
	Product ID	(Foreign key)	
	Personnr.	(Foreign key)	

Columns/Data Types for Order:

	Name	Data Type	Data Type Size
	Order ID	VARCHAR(n)	15
	Payment Reference	VARCHAR(n)	25
	Order_Date	DATE	
	Date: Last Change	DATE	
	Status	CHAR(n)	15
	Tracking Nr.	VARCHAR(n)	35
	Personnr.	(Foreign key)	

Columns/Data Types for Product:

	Name	Data Type	Data Type Size
	Product ID	VARCHAR(n)	8
	Product Title	VARCHAR(n)	30
	Description	VARCHAR(n)	700
	Short Description	VARCHAR(n)	70
	Quantity In Stock	INT	
	Discount %	INT	
	Retail Price w/o VAT	FLOAT	
	VAT	FLOAT	
	Link	VARCHAR(n)	120
	Is Featured	Custom	BOOLE...
	Department ID	(Foreign key)	

Columns/Data Types for ProductsInOrder:

	Name	Data Type	Data Type Size
	Order ID	(Foreign key)	
	Product ID	(Foreign key)	
	Quantity	INT	

Columns/Data Types for Keyword:

	Name	Data Type	Data Type Size
	Word	VARCHAR(n)	15

Columns/Data Types for Department:

	Name	Data Type	Data Type Size
	Department ID	INT	
	Department Title	VARCHAR(n)	25
	Description	VARCHAR(n)	70
	Link	VARCHAR(n)	120

All other tables only contain foreign keys and are therefore not represented here.

Milestone 3:

Python Script for Tasks 3, 4 & 5:

<https://drive.google.com/file/d/1tXdJ0Zw9c3ZrN3IPU4stnB9bEHcAOXti/view?usp=sharing>

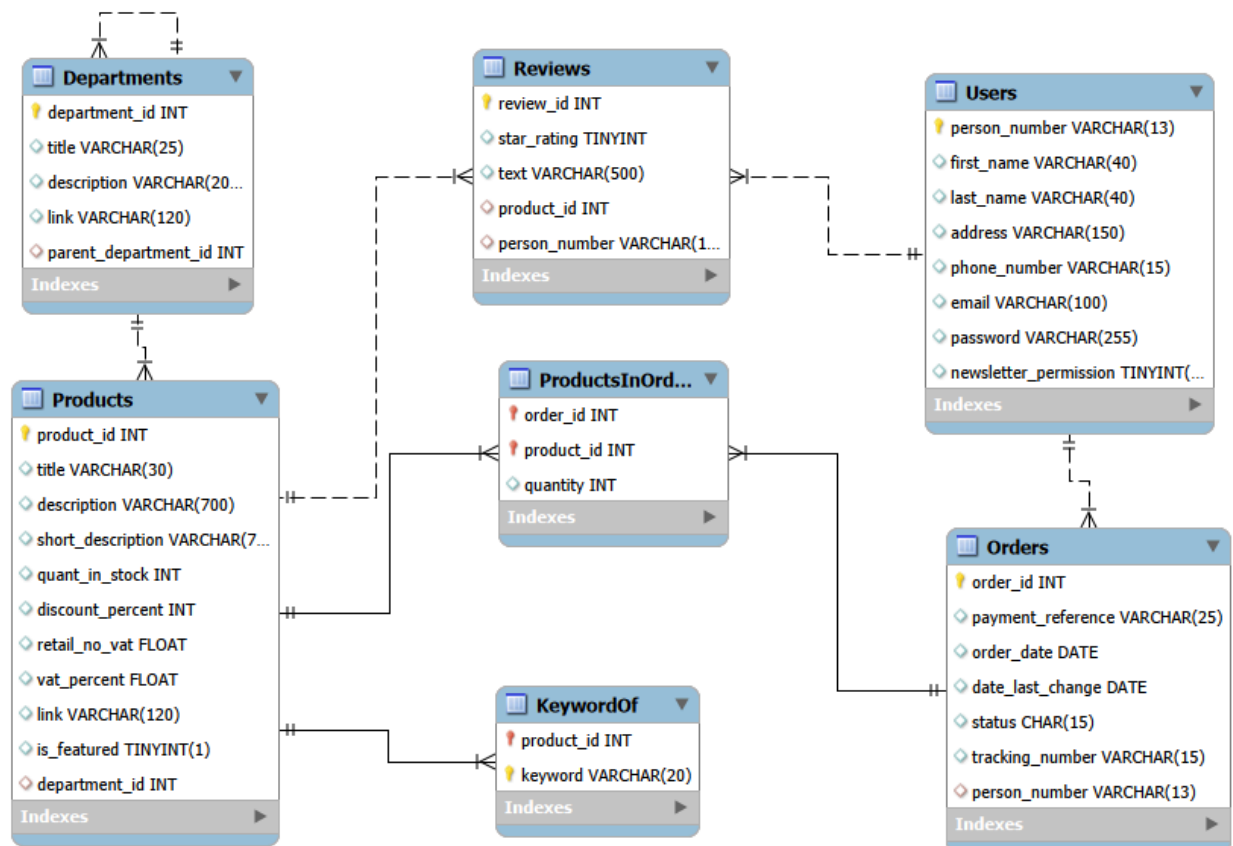
All SQL commands for creating and populating tables, as well as querying, can be viewed in this python script.

SQL queries to create and populate tables are saved as python strings with variable names like create_something or add_something

SQL queries to search the database have names like query_to_do_something.

Database Schema:

Below you can see the database schema that was obtained through Workbench's Reverse Engineer.



Populating The Tables

We populated the tables successfully using the basic SQL commands, as you can see from the tables.

department...	title	description	link	parent_department_id
1	Home	Welcome to AltOnline	/home	NULL
2	Electronics	Electronic devices	/electronics	1
3	Laptops	All kinds of laptops	/electronics/laptops	2
4	Phones	Smartphones and mobile phones	/electronics/phones	2
6	Books	Our store offers tons of awesome books.	/home/books	1
NULL	NULL	NULL	NULL	NULL

We then inserted the products

product...	title	description	short_description	quant_in_stock	discount_percent	retail_no_vat	vat_percent	link	is_featured	department...
1	iPhone 7	epic iphone super cool...	cool phone	2000	5	12000	25	www.altonline.com/iphone7	1	4
2	iPhone 6	sorta cool iphone helps...	less cool phone	1000	12	8000	25	www.altonline.com/iphone6	0	4
3	Pippi Lå...	Astrid Lindgren's classi...	pippi är blåst	10	0	200	0	www.altonline.com/pippilangstrump	0	6
11	Macboo...	epic laptop super cool...	cool laptop	2000	5	30000	25	www.altonline.com/macbookpro	1	4
12	Macboo...	it's a macbook but it ru...	slow laptop	100	10	10000	25	www.altonline.com/macbookslow	0	4
13	Of Mice...	Steinbeck was the gooaat	is it about mice? or men? or both???	120	0	100	0	www.altonline.com/ofmiceandmen	0	6
14	Of Rice...	A practical guide for ric...	is it about rice? or men? or both???	120	0	100	0	www.altonline.com/ofriceandmen	0	6
15	Element...	Swedish for absolute b...	A basic swedish book	120	0	150	0	www.altonline.com/elementaryswedish	0	6
16	Clown C...	Have you ever been int...	A basic clown book	120	0	600	0	www.altonline.com/clownclass	1	6
17	Advance...	ONLY FOR EXPERTS:...	An advanced clown book	120	0	800	0	www.altonline.com/advancedclowning	1	6
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Followed by Keywordof

	product_id	keyword
▶	11	Macbook
	12	Macbook
	13	Men
	14	Men
*	NULL	NULL

Orders

	order_id	payment_reference	order_date	date_last_change	status	tracking_number	person_number
▶	1	13524524	2025-12-01	2025-12-02	In transit	NULL	199901170327
	2	13524524	2025-12-01	2025-12-02	In transit	NULL	199901170327
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ProductsInOrder

	order_id	product_id	quantity
▶	1	17	2
*	NULL	NULL	NULL

Reviews

	review_id	star_rating	text	product_id	person_number
▶	1	5	Great product, very fast.	1	199901170327
	2	4	Good value for money.	1	200301170327
*	NULL	NULL	NULL	NULL	NULL

And, finally, Users

	person_number	first_name	last_name	address	phone_number	email	password	newsletter_permission
▶	199901170327	Goran	Johansson	Sveasvängen 3, Norrköping	696969696	svea2345@svea.com	Johansson_är_rolig	1
	200301170327	Alfons	Foks	Gubbegatan 3, Norrköping	420420420	alfons5@svea.com	tyckerintetomfisk	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

SQL Queries

In the following paragraph we will go over how we wrote our SQL queries. The implementation for all of the following queries can be found in the [Python code](#) at the top of Milestone 3. For the more difficult queries we also added our explanations.

- Welcome text for the homepage

```
SELECT description, department_id FROM Departments
WHERE department_id = 1
```

- List of the top level departments with fields needed for the homepage

```
SELECT * FROM Departments
WHERE parent_department_id = 1
```

- List of the featured products with fields needed for the homepage

```
SELECT * FROM Products
WHERE is_featured
```

- Given a product, list all keyword-related products

```
SELECT * FROM Products
WHERE product_id IN(
    SELECT product_id FROM KeywordOf
    WHERE keyword IN (
        SELECT keyword FROM KeywordOf
        WHERE product_id = 11
    )
)
```

For the keyword-related products (query_to_get_keyword_relateds), we perform 3 nested queries:

The inner one finds all keywords related to a given product_id

The intermediate finds all product_ids related to a set of keywords

The outer one finds the rest of the product fields related to those product_ids.

This implementation does have the quirk of returning the original product itself. This could be easily remedied with a final WHERE statement. But logically, there is no problem with considering an item keyword-related to itself.

- Given a department, list of all its products (title, short description, current retail price) with their average rating

```
WITH CURRENT_PRICES AS(  
SELECT product_id, ((retail_no_vat - discount_percent/100)*(1 +  
vat_percent/100))AS current_price FROM Products  
) ,
```

```
AVERAGE_RATINGS AS(  
SELECT product_id, AVG(star_rating) AS avg_rating FROM Reviews  
GROUP BY product_id)
```

```
SELECT Products.title, Products.short_description,  
CURRENT_PRICES.current_price, AVERAGE_RATINGS.avg_rating  
FROM Products  
LEFT JOIN CURRENT_PRICES ON (Products.product_id =  
CURRENT_PRICES.product_id)  
LEFT JOIN AVERAGE_RATINGS ON (Products.product_id =  
AVERAGE_RATINGS.product_id)  
WHERE Products.department_id = 4
```

For the query to get all products from a (leaf) department along with some derived attributes of those products, we build two intermediate tables, to join using our final query. We build a current prices table linking each product_id with price, then build an average ratings table using the AVG() aggregation function on the reviews table, grouping by product_id. Many product id will have no entry because no reviews have been left. Finally, we join the product table to those new tables on product_id, left joining so that every product_id has entries for those right-table columns. Finally, we use a where to only select for the department we care about.

- List of all products on sale sorted by the discount percentage (starting with the biggest discount)

```
SELECT * FROM Products  
WHERE discount_percent > 0  
ORDER BY discount_percent DESC
```

Performance Analysis

Using the EXPLAIN keyword before queries, we read the number of rows estimated to be scanned. For complex queries, EXPLAIN may return multiple results related to each subquery. In these cases, we add up the number of rows from each result.

Before adding indices, these are the number of rows that EXPLAIN estimates the query will scan:

welcome text: 1,
toplevel depts: 2,
products of leaf dept: 9,
products on sale sort: 10,
Featured products: 10,
Keyword related: 6

We added a B-tree (default) indices to the table Products on *discount_percent* and *is_featured*.

Adding the index on *discount_percent* reduced the number of rows scanned by the query “products on sale sort” from 10 to 4.

Adding the index on *is_featured* reduced the number of rows scanned by the query “featured products” from 10 to 5.

Hash indices are ideal for finding individual records from a value of a certain column because they give $O(1)$ lookups. This could be perfect for optimizing queries such as “which items have a price of exactly 1000 kr” or “give me all the details of all customers named Lars.”

In contrast, B-tree indices give $O(\log(N))$ lookup times but they have the added benefit of preserving the data’s sorting order in the structure of the tree. This is ideal for queries involving sorting or inequalities. For example, if one queries the database for products costing less than a certain amount, the B-tree index can help the database realize that a certain branch of the tree is composed of products costing above some greater amount. If this is the case, the database can ignore that branch of the tree.

Milestone 4:

The scripts to let a client view/update the database are down below, and in the submission folder. To set it up on your machine, ensure you have all the necessary packages installed. We use the dotenv package to manage passwords. (pip install python-dotenv)

Add a .env file with the following contents to support authentication.

```
SSH_USERNAME=  
SSH_PASSWORD=  
DB_USER=  
DB_PASSWORD=
```

It must be saved as “.env” in the same directory that you run the scripts from.

Each script asks the user for inputs, then runs SQL queries from those inputs.

The department search begins by searching for a department with the given id. If none exists, that is reported to the user. Otherwise, the script runs a query to check whether the department has children. If so, it lists them. If not, then the department is a leaf department, so the script runs a query to find all products belonging to that department along with their computed retail prices with discount, listing each product's relevant fields.

The product search begins by asking the user for a Product_ID(from Products). It then retrieves and displays the current discount_percent for that product. It then asks the user if they want to change the discount. If the user confirms, it asks for a new discount value, validates that it is a number and runs an UPDATE statement on Products.discount_percent. The program commits the change and then shows the updated discount to confirm. This fulfils the requirement to show the current discount and allow the user to modify it.

The code for the *department_search.py*

```
import os  
import pymysql  
from sshtunnel import SSHTunnelForwarder  
import getpass
```

```

from dotenv import load_dotenv

# Load environment variables from .env if present
load_dotenv()

# Read credentials from environment with fallback to interactive prompts
# Environment variable names used: SSH_USERNAME, SSH_PASSWORD, DB_USER,
DB_PASSWORD
ssh_username = os.getenv('SSH_USERNAME') or input("Enter your SSH
username: ")
ssh_password = os.getenv('SSH_PASSWORD') or getpass.getpass("Enter your
SSH password: ")

db_user = os.getenv('DB_USER') or input("Enter your group name: ")
db_password = os.getenv('DB_PASSWORD') or getpass.getpass("Enter your
group password: ")

# Set SSH tunnel
tunnel = SSHTunnelForwarder(
    ('fries.it.uu.se', 22), #port = 22
    ssh_username=ssh_username,
    ssh_password=ssh_password,
    remote_bind_address=('127.0.0.1', 3306)
)

# Start SSH tunnel
tunnel.start()

# Connect to MySQL database
mydb = pymysql.connect(
    host='127.0.0.1',
    user=db_user,
    password=db_password,
    port=tunnel.local_bind_port,
    database=db_user
)

# Excute MySQL statements
mycursor = mydb.cursor()

def run_client():
    dept_to_get = str(int(input("Which department_id do you want
information about? Enter a number: "))) #int should basically sanitize
input

```

```

    get_dept_query = f"SELECT * FROM Departments WHERE department_id =
{dept_to_get}"
    mycursor.execute(get_dept_query)
    this_dept = list(mycursor)
    if len(this_dept) == 0:
        print("This department id does not exist.")
        return

    get_children_query = f"SELECT * FROM Departments WHERE
parent_department_id = {dept_to_get}"
    mycursor.execute(get_children_query)
    children = list(mycursor)

    if len(children) == 0:
        get_products_query = f"""
                                WITH RETAIL_PRICES AS (
                                    SELECT product_id, (retail_no_vat -
discount_percent/100) AS retail_price FROM Products
                                )
                                SELECT Products.product_id, Products.title,
Products.short_description, RETAIL_PRICES.retail_price
                                FROM Products
                                LEFT JOIN RETAIL_PRICES ON
(Product.product_id = RETAIL_PRICES.product_id)
                                WHERE Products.department_id = {dept_to_get}
                                """

        mycursor.execute(get_products_query)
        prods = list(mycursor)
        for prod in prods:
            print(f"ID:{prod[0]}, Title:{prod[1]}, Retail Price after
discount:{prod[3]} SEK")
        else:
            print("Child Departments:")
            for child in children:
                print(f"ID:{child[0]}, Title:{child[1]}")

run_client()

# Close MySQL and SSH connections
mydb.close()
tunnel.stop()

```

The code for *product_search.py*:

```
import os
import pymysql
from sshtunnel import SSHTunnelForwarder
import getpass
from dotenv import load_dotenv

# Load environment variables from .env if present
load_dotenv()

# Read credentials from environment with fallback to interactive prompts
# Environment variable names used: SSH_USERNAME, SSH_PASSWORD, DB_USER,
# DB_PASSWORD
ssh_username = os.getenv('SSH_USERNAME') or input("Enter your SSH
username: ")
ssh_password = os.getenv('SSH_PASSWORD') or getpass.getpass("Enter your
SSH password: ")

db_user = os.getenv('DB_USER') or input("Enter your group name: ")
db_password = os.getenv('DB_PASSWORD') or getpass.getpass("Enter your
group password: ")

# Set SSH tunnel
tunnel = SSHTunnelForwarder(
    ('fries.it.uu.se', 22), #port = 22
    ssh_username=ssh_username,
    ssh_password=ssh_password,
    remote_bind_address=('127.0.0.1', 3306)
)

# Start SSH tunnel
tunnel.start()

# Connect to MySQL database
mydb = pymysql.connect(
    host='127.0.0.1',
    user=db_user,
    password=db_password,
    port=tunnel.local_bind_port,
    database=db_user
)

# Excute MySQL statements
```

```
mycursor = mydb.cursor()
```

```
def run_client():
    prod_to_get = str(int(input("Which product_id do you want to know the
discount of? Enter a product_id: "))) #int should basically sanitize input

    get_prod_query = f"SELECT * FROM Products WHERE product_id =
{prod_to_get}"
    mycursor.execute(get_prod_query)
    this_prod = list(mycursor)
    if len(this_prod) == 0:
        print("This product id does not exist.")
        return
    else:
        print(f"Current discount: {this_prod[0][5]} %")
        change_value = str(input("Change this percentage? Enter an int to
change it to, or 'N' for no change: "))
        if change_value == "N":
            return
        else:
            try:
                change_to = int(change_value)
            except:
                print("Input was not a int or 'N.' Quitting...")
                return
            set_discount_query = f"UPDATE Products SET discount_percent =
{change_to} WHERE product_id = {prod_to_get}"
            try:
                mycursor.execute(set_discount_query)
                mydb.commit()
                print("The discount percentage has been updated")
            except:
                print("Your database update failed")
            return
```

```
run_client()
```

```
# Close MySQL and SSH connections
mydb.close()
tunnel.stop()
```